



ADOBE EXPERIENCE MANAGER

Algolia AEM Extensions

Reference indexing extensions for PDF text and tags, and a guide to building page and asset request extenders for the Algolia Connector for AEM.

VERSION

1.0.0

PUBLISHED

September 29, 2026

CONTENTS

Installation · PDF Text Extractor · Tags Extractor ·
Building Custom Extensions

Contents

PART 01 Installation Guide

Prerequisites, building the extensions package, deploying it to AEM, and running tests.

Introduction

Supported versions and runtimes

Before you start

Modules

Build and install

Select the extenders

Testing

Maven settings

PART 02 PDF Text Extractor

Extracting PDF text into Algolia records, the word-size limit, and splitting records that exceed the size threshold.

When it runs

How text is stored

Record splitting

Configuration

Dependencies

PART 03 Tags Extractor

Adding cq:tags from pages and assets onto Algolia records with the default tags extractor.

Registration

What it writes

Page and asset entry points

PART 04 Building Custom Extensions

Implementing page and asset request extenders, registering them as OSGi services, and selecting them on a Cloud Service configuration.

Choose an interface

Register the OSGi service

Augment the request

Compile against the indexer

Ship and select the service

Installation Guide

Prerequisites, building the extensions package, deploying it to AEM, and running tests.

How to build and install the **Algolia AEM Extensions** package (`algolia-aem-extensions.all`) so the reference indexing extenders are available to the Algolia Connector for Adobe Experience Manager.

This package does not index content by itself. Install and configure `algolia-aem-indexer` first. The extenders in this project run during the indexer's page and asset indexing and depend on services the indexer provides.

No warranties or SLA. This project is a reference implementation, provided as-is. There is no service level agreement and no guarantee of support. Use it as a starting point and adapt it to your requirements.

Introduction

The extensions package ships two OSGi services that augment Algolia records while the indexer prepares them:

Extender	Applies to	What it adds
PDF Text Extractor	Assets	Text extracted from PDF renditions, split across attributes or records when the text is large
Tags Extractor	Pages and assets	Tags read from the <code>cq:tags</code> property

Both are registered as request extenders. After you install the package, select them on the indexer Cloud Service configuration. See [Building custom extensions](#) if you need a different payload.

Supported versions and runtimes

AEM version	6.5+ (AEM SDK API <code>2025.12</code>)
Java	11 or newer, for the Maven build
Indexer dependency	<code>algolia-aem-indexer.core</code> 4.1.2

The PDF extractor also embeds Apache OpenNLP `opennlp-tools` 2.5.0, used to split extracted text on whitespace.

Before you start

1. Install `algolia-aem-indexer.all` and upload a valid Algolia license. Indexing stays inactive until the license is in place.
2. Create an indexer Cloud Service configuration and assign it to the sites or DAM folders you index.
3. Host `algolia-aem-indexer.core` where this project's Maven build can resolve it. The core module declares it as a compile dependency:

```
<dependency>
  <groupId>com.algolia</groupId>
  <artifactId>algolia-aem-indexer.core</artifactId>
  <version>4.1.2</version>
</dependency>
```

The indexer project documents how to install that JAR in a local or private Maven repository.

You need Maven 3.3.9 or newer.

Modules

Module	Artifact	Role
core	<code>algolia-aem-extensions.core</code>	OSGi bundle with <code>DefaultAlgoliaPdfTextExtractor</code> and <code>DefaultAlgoliaTagsExtractor</code>
all	<code>algolia-aem-extensions.all</code>	Container content package. Embeds the core bundle at <code>/apps/algolia-extensions-packages/application/install</code>

An `it.tests` module is present for HTTP integration tests against a running AEM instance. It is not part of the reactor, so `mvn clean install` from the project root does not run it.

Build and install

From the project root:

```
mvn clean install
```

That builds the core bundle and the `all` package. The installable file is:

```
all/target/algolia-aem-extensions.all-1.0.0-SNAPSHOT.zip
```

Deploy to a local author

```
mvn clean install -PautoInstallSinglePackage
```

Publish instance:

```
mvn clean install -PautoInstallSinglePackagePublish
```

Or point the author install at another port:

```
mvn clean install -PautoInstallSinglePackage -Daem.port=4503
```

Bundle only, to the author:

```
mvn clean install -PautoInstallBundle
```

A single content package, run from that module (for example `all`):

```
mvn clean install -PautoInstallPackage
```

Default connection settings are `localhost:4502` (author) and `localhost:4503` (publish), with user `admin` / `admin`. Override them with `aem.host`, `aem.port`, `aem.publish.host`, `aem.publish.port`, `sling.user`, `sling.password`, `vault.user`, and `vault.password`.

Select the extenders

Installing the package registers the services. Indexing does not call them until a Cloud Service configuration selects them.

On the indexer Cloud Service configuration **Advanced** tab:

Dropdown	Service to select	Label
Asset Request Extender Service	<code>com.algolia.core.extender.internal.DefaultAlgoliaPdfTextExtractor</code>	Algolia PDF Text Extractor

Dropdown	Service to select	Label
Asset Request Extender Service	<code>com.algolia.core.extender.internal.DefaultAlgoliaTagsExtractor</code>	Algolia Default Tags Extractor
Page Request Extender Service	<code>com.algolia.core.extender.internal.DefaultAlgoliaTagsExtractor</code>	Algolia Default Tags Extractor

The dropdown label is the OSGi `service.description` when the component sets one, and the component name otherwise. The stored value is the implementation class name.

Reindex or publish a page or PDF asset that uses the configuration, then confirm the new attributes on the Algolia record. See [PDF Text Extractor](#) and [Tags Extractor](#) for the attribute names.

Testing

Unit tests

```
mvn clean test
```

The core module tests the PDF extractor (mime type, empty text, word-limit splits, the 10 KB record split) and the tags extractor (page and asset paths both call `TagsParserService`).

Integration tests

`it.tests` talks to a running AEM instance over HTTP. It is not a reactor module. From `it.tests`:

```
mvn clean verify -Plocal
```

Test classes live under `src/main/java` and must match `*IT.java`.

Property	Description	Default
<code>it.author.url</code>	Author URL	<code>http://localhost:4502</code>
<code>it.author.user</code>	Author user	<code>admin</code>
<code>it.author.password</code>	Author password	<code>admin</code>
<code>it.publish.url</code>	Publish URL	<code>http://localhost:4503</code>
<code>it.publish.user</code>	Publish user	<code>admin</code>
<code>it.publish.password</code>	Publish password	<code>admin</code>

Maven settings

The project expects Adobe's public Maven repository. To add it to your settings, follow [Set up the Adobe Maven repository](#).

Related docs

Doc	Topics
PDF Text Extractor	<code>pdfText</code> attributes, word limit, record splitting
Tags Extractor	<code>cq:tags</code> on pages and assets
Custom extensions	Writing and registering your own extender

PDF Text Extractor

Extracting PDF text into Algolia records, the word-size limit, and splitting records that exceed the size threshold.

`DefaultAlgoliaPdfTextExtractor` is an asset request extender. When the indexer prepares a record for a PDF asset, this service reads the document text and writes it onto the Algolia record so the PDF body is searchable.

Source: `DefaultAlgoliaPdfTextExtractor`.

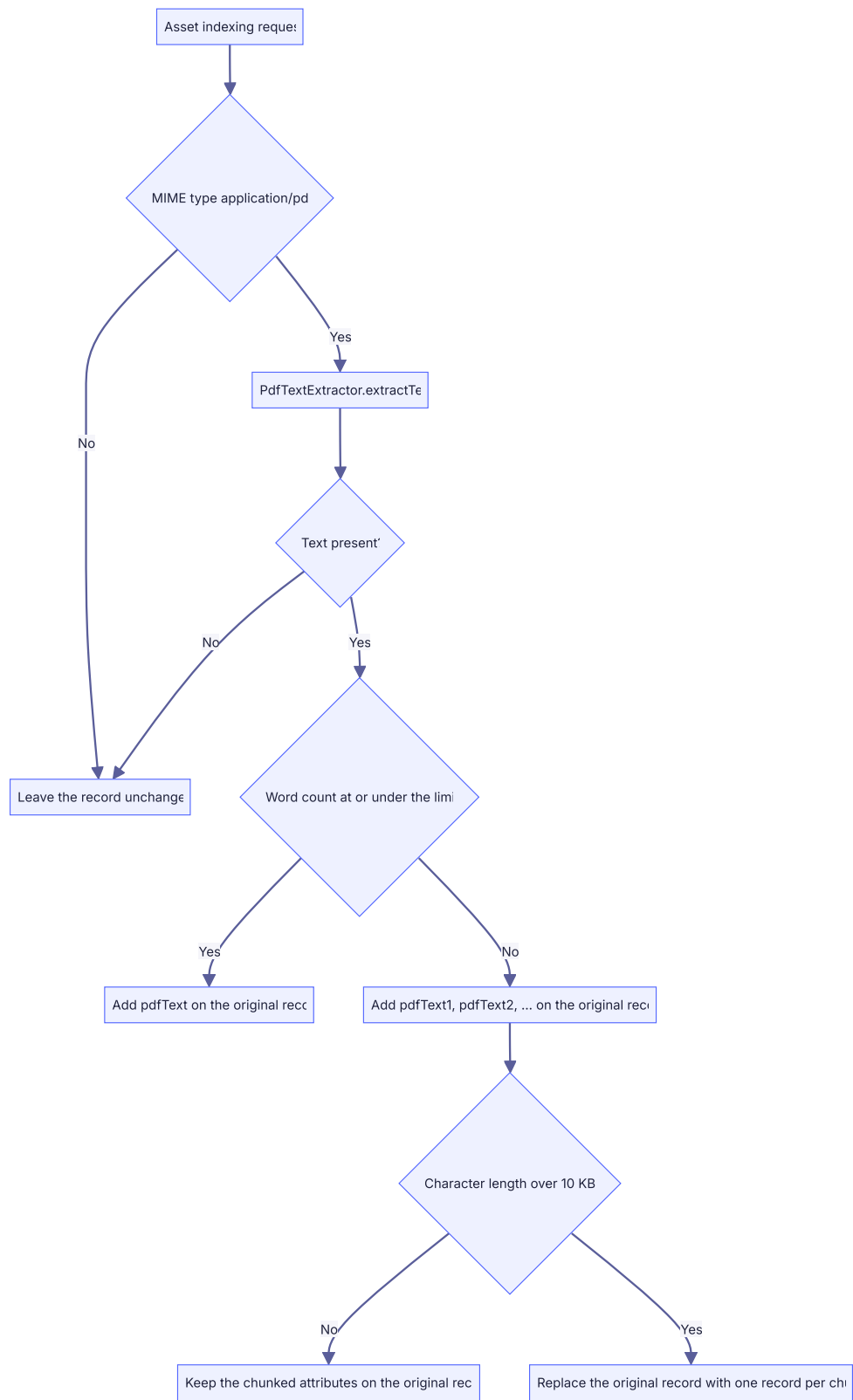
When it runs

The component is an OSGi service named **Algolia PDF Text Extractor**. It implements

`AlgoliaAssetRequestExtender` only, so it appears in the **Asset Request Extender Service** dropdown and not in the page dropdown.

`augmentAlgoliaRequest` runs after the indexer has created the asset's `AlgoliaRequest`. The method returns immediately unless the asset MIME type is `application/pdf`. Non-PDF assets, including a null MIME type, are left unchanged.

For a PDF, it calls the indexer's `PdfTextExtractor.extractText`. Empty or null text is ignored. The original record is not given a `pdfText` attribute in that case.



How text is stored

Words are counted with OpenNLP `WhitespaceTokenizer`. The limit is the OSGi **Word Size Limit** (default **900**).

Extracted text	Attributes written	Records
Word count $\leq$ limit	One attribute, <code>pdfText</code> , holding the full string	The original asset record
Word count $>$ limit, and the string is 10 KB or shorter	<code>pdfText1</code> , <code>pdfText2</code> , ... each holding up to the word limit	Still the original asset record
Word count $>$ limit, and the string is longer than 10 KB	Each chunk becomes the <code>pdfText</code> attribute of its own record	The original record is removed. See Record splitting

A chunk is the word limit words, joined with single spaces. The last chunk holds the remainder. Attribute numbering starts at `pdfText1`. There is no bare `pdfText` attribute once the word count exceeds the limit.

Example with the default limit of 900 and 1,801 words, while the string stays under 10 KB: the original record gains `pdfText1` (900 words), `pdfText2` (900 words), and `pdfText3` (1 word).

The 10 KB check uses `String.length()` on the full extracted text, not the size of the finished Algolia JSON record. The constant is `RECORD_SIZE_LIMIT` (`10 * 1024`). Text that is exactly 10 KB is not split into extra records.

Record splitting

Splitting runs only when both are true: the word count is over the limit, and `text.length()` is greater than 10 KB.

For each attribute whose name starts with `pdfText`, the extender:

1. Builds a new `AlgoliaRecord` whose object ID is `{originalObjectID}_{index}`, starting at `0`.
2. Copies that chunk onto the new record as `pdfText` (the same value, under the name `pdfText`).
3. Sets `path` to the asset path (`AlgoliaConstants.ATTRIBUTE_PATH`).
4. Adds the record to the request.

It then removes the original record (the one whose object ID was captured before splitting). Other attributes that the indexer had placed on the original record are not copied onto the split records. Each split record contains the object ID, `pdfText`, and `path`.

After the swap it calls `AlgoliaUtil.handleSplitRecordCount` with `SplitRecordAction.ADD`, the asset path, the number of remaining records, and the `ResourceResolverFactory`. It also sets `AlgoliaRequest.setPdfTextSplittingAttempted(true)` so the indexer can tell this path was taken.

Configuration

The metatype is **DefaultAlgoliaPdfTextExtractor Configuration**.

Property	OSGi id	Default	Meaning
Word Size Limit	<code>word.size.limit</code>	900	Maximum words stored in a single <code>pdfText</code> attribute before the text is partitioned

The component declares the default in code (`word_size_limit() default 900`) and in the OSGi component descriptor. Change it with an OSGi configuration for PID `Algolia PDF Text Extractor` (the component `name`), property `word.size.limit` .

A lower limit produces more `pdfTextN` attributes and, once the full string exceeds 10 KB, more Algolia records. A higher limit keeps more text on fewer attributes and can produce a record Algolia will reject if a single attribute or the record exceeds Algolia's size limits. 900 words is the shipped default; validate record size against your index settings before raising it.

Dependencies

Service or library	Use
<code>PdfTextExtractor</code>	Reads text from the DAM <code>Asset</code> . Provided by the indexer.
<code>ResourceResolverFactory</code>	Passed to <code>AlgoliaUtil.handleSplitRecordCount</code> when records are split
<code>opennlp-tools</code> 2.5.0	<code>WhitespaceTokenizer</code> for the word count and the partitions

The extractor logs `Encountered PDF asset, extracting text from it.` at info when it sees a PDF.

Related docs

Doc	Topics
Installation	Build, deploy, and select the extender
Tags Extractor	The page and asset tags extender
Custom extensions	The <code>AlgoliaAssetRequestExtender</code> contract

Tags Extractor

Adding cq:tags from pages and assets onto Algolia records with the default tags extractor.

`DefaultAlgoliaTagsExtractor` copies AEM tags onto the Algolia record for both pages and assets. It reads the JCR `cq:tags` property and lets the indexer's `TagsParserService` turn those tag IDs into record attributes.

Source: `DefaultAlgoliaTagsExtractor`.

Registration

The class implements both extender interfaces and registers both OSGi services:

```
@ComponentServiceProperties(description = "Algolia Default Tags Extractor")
@Component(
    service = {
        AlgoliaPageRequestExtender.class,
        AlgoliaAssetRequestExtender.class
    }
)
public class DefaultAlgoliaTagsExtractor
    implements AlgoliaPageRequestExtender, AlgoliaAssetRequestExtender {
```

`@ComponentServiceProperties` sets `service.description` to **Algolia Default Tags Extractor**. That string is the label in the Cloud Service dropdowns. The value stored on the configuration is the class name `com.algolia.core.extender.internal.DefaultAlgoliaTagsExtractor`.

Select it independently for pages and for assets:

Dropdown	Select it when
Page Request Extender Service	Indexed pages should carry their <code>cq:tags</code>
Asset Request Extender Service	Indexed assets should carry their <code>cq:tags</code>

One bundle installation covers both. Choosing it on only one dropdown limits it to that resource type.

What it writes

Both entry points call the same method:

```
private void addTagsToAlgoliaRecord(AlgoliaRequest request) {  
    Resource resource = request.getResource();  
    AlgoliaRecord algoliaRecord = request.getAlgoliaRecords().get(0);  
    this.tagsParserService.parse(resource, algoliaRecord, PN_TAGS);  
}
```

`PN_TAGS` is `com.day.cq.tagging.TagConstants.PN_TAGS`, the property name `cq:tags`.

Behavior that follows from this method:

- Tags are read from the resource already on the `AlgoliaRequest`, not by adapting the `Page` or `Asset` argument.
- Only the **first** record on the request is updated (`getAlgoliaRecords().get(0)`). Later records, including records created by the [PDF Text Extractor](#), are not given tags by this class.
- The shape of the attributes is defined by `TagsParserService` in the indexer, not by this class. This extender does not rename tags or apply a locale.

If the request has no records, `get(0)` throws. The indexer is expected to have created the initial record before extenders run.

Page and asset entry points

```
@Override  
public void augmentAlgoliaRequest(AlgoliaRequest request, Asset asset) {  
    this.addTagsToAlgoliaRecord(request);  
}  
  
@Override  
public void augmentAlgoliaRequest(AlgoliaRequest request, Page page) {  
    this.addTagsToAlgoliaRecord(request);  
}
```

The `Page` and `Asset` arguments are unused. Two methods exist so the class satisfies both interfaces and can be selected in both dropdowns. A custom extender that needs the page title, the asset MIME type, or a property that is not on `request.getResource()` should use those arguments. See [Building custom extensions](#).

The only injected service is `TagsParserService`.

Related docs

Doc	Topics
Installation	Build, deploy, and select the extender
PDF Text Extractor	Asset-only text extraction and record splitting
Custom extensions	<code>@ComponentServiceProperties</code> and the extender interfaces

Building Custom Extensions

Implementing page and asset request extenders, registering them as OSGi services, and selecting them on a Cloud Service configuration.

The two classes in this project are reference implementations. Copy their shape when you need fields the indexer does not add on its own: a computed attribute, a transformed tag title, or extra records for a large payload.

The indexer invokes every extender selected on the Cloud Service configuration **Advanced** tab while it builds the indexing payload. Your service can add, change, or remove attributes, and it can add records. It cannot replace the indexer's decision to index the resource; aborting indexing is a separate introspector hook, documented in the indexer project.

Choose an interface

Interface	Called for	Method
<code>AlgoliaPageRequestExtender</code>	Pages	<code>augmentAlgoliaRequest(AlgoliaRequest request, Page page)</code>
<code>AlgoliaAssetRequestExtender</code>	Assets	<code>augmentAlgoliaRequest(AlgoliaRequest request, Asset asset)</code>

Implement one interface, or both when the same logic applies to pages and assets.

`DefaultAlgoliaTagsExtractor` implements both and delegates to one private method.

`DefaultAlgoliaPdfTextExtractor` implements only the asset interface, because PDF text does not apply to pages.

Both interfaces are `@ConsumerType`. The indexer calls `augmentAlgoliaRequest` after it has created the `AlgoliaRequest` and its initial record.

Register the OSGi service

Declare the service interfaces explicitly when a class implements more than one, so OSGi publishes each contract:

```
@ComponentServiceProperties(description = "My page and asset extender")
@Component(
    service = {
```

```

        AlgoliaPageRequestExtender.class,
        AlgoliaAssetRequestExtender.class
    }
}

public class MyRequestExtender
    implements AlgoliaPageRequestExtender, AlgoliaAssetRequestExtender {

```

`@ComponentServiceProperties` is an OSGi component property type from the indexer (`PREFIX_` is `service.`). Its `description` becomes `service.description` .

The Cloud Service dropdown uses that description as the option label. If `service.description` is empty, the dropdown falls back to the component name (`@Component(name = "...")`). The option **value** stored on the configuration is the implementation class name, not the label.

`DefaultAlgoliaPdfTextExtractor` does not set `service.description` . Its dropdown label is the component name **Algolia PDF Text Extractor**. Prefer `@ComponentServiceProperties` for a label you control independently of the component name.

A single-interface component can rely on Declarative Services to publish the implemented interface:

```

@ComponentServiceProperties(description = "My asset extender")
@Component
public class MyAssetExtender implements AlgoliaAssetRequestExtender {

```

Inject indexer or AEM services with `@Reference` on the constructor, as both reference implementations do.

Optional configuration uses an OSGi `@ObjectClassDefinition` and a constructor parameter of that type, the same pattern as the PDF extractor's `word_size_limit` .

Augment the request

```

@Override
public void augmentAlgoliaRequest(AlgoliaRequest request, Page page) {
    AlgoliaRecord record = request.getAlgoliaRecords().get(0);
    record.addAttribute("contentType", "article");
}

```

Guidelines that match the shipped extenders:

- Read the resource from `request.getResource()` when the JCR node is what you need. Use the `Page` or `Asset` argument for API that is not on that resource (template, MIME type, renditions).
- Update the record you mean to update. The tags extractor only touches `getAlgoliaRecords().get(0)` . The PDF extractor adds further records with `request.addRecord` and then removes the original.
- Keep each record within Algolia's size limits. The PDF extractor treats 10 KB of extracted text as the point where one asset becomes many records, and it names the new object IDs `{originalObjectID}_{index}` .

- Do not assume your extender is the only one selected. Another extender may have already changed the first record, or may run after you. Order is the indexer's service binding order, not something this package configures.
- Leave the request unchanged when your input is absent (the PDF extractor ignores non-PDFs and empty text) so unrelated resources still index.

Compile against the indexer

Add the indexer core JAR to the bundle's Maven dependencies. This project uses:

```
<dependency>
  <groupId>com.algolia</groupId>
  <artifactId>algolia-aem-indexer.core</artifactId>
  <version>4.1.2</version>
</dependency>
```

Packages you will import:

Package	Types
com.algolia.connector.core.extender	AlgoliaPageRequestExtender, AlgoliaAssetRequestExtender
com.algolia.connector.core.domain	AlgoliaRequest, AlgoliaRecord
com.algolia.connector.core.annotation	ComponentServiceProperties
com.algolia.connector.core	PdfTextExtractor, TagsParserService, AlgoliaConstants

`algolia-aem-indexer.core` must be on the Maven classpath at build time and installed in AEM at runtime. The extender bundle does not embed the indexer.

Put new classes in the `core` module, next to the reference implementations, or in your own bundle that imports the same packages. The `all` package embeds `algolia-aem-extensions.core` only. A separate bundle needs its own content package or an extra `<embedded>` entry.

Ship and select the service

1. Build and deploy with the commands in [Installation](#).
2. Confirm the component is active in the OSGi console and that it is registered under the extender interface you implemented.
3. Open the indexer Cloud Service configuration, **Advanced** tab, and select the service in **Page Request Extender Service**, **Asset Request Extender Service**, or both.
4. Publish or reindex a resource that uses that configuration.

5. In Algolia, open the record and confirm the attributes you added. For a split PDF, expect several object IDs derived from the asset path.

Unit-test the extender by constructing it with mocks for its `@Reference` services and an `AlgoliaRequest` that already contains one record. The tests under `core/src/test/java/com/algolia/core/extender/internal/` cover the PDF and tags classes that way.

Related docs

Doc	Topics
Installation	Prerequisites, Maven build, deploy profiles
PDF Text Extractor	Asset extender with OSGi configuration and record splitting
Tags Extractor	Dual page and asset registration